

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:37:01.9095221Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-083702034.prof
Profil mémoire	memory-20260924-083702034.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	143.19 ms	143190812	0	0

Détail hyperfine

Métrique	Valeur
Moyenne	2.8189 s
Écart-type	n/a
Médiane	2.8189 s
Temps utilisateur	1.5575 s
Temps système	0.9784 s
Min	2.8189 s
Max	2.8189 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	1.22s
Mémoire allouée (pprof)	19812.81kB
Mémoire par opération	0 B/op

Garbage collector Go

Cycles GC	55
Temps GC cumulé	63.840 ms

Plus longue pause	5.820 ms
Temps CPU GC cumulé	322.560 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **1.22s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	1.01s	82.79%	1.17s	95.90%
jeu-de-la-vie/game.(*Board).index (inline)	0.10s	8.20%	0.10s	8.20%
jeu-de-la-vie/game.squaredDistance (inline)	0.06s	4.92%	0.06s	4.92%
runtime.cgocall	0.03s	2.46%	0.03s	2.46%
jeu-de-la-vie/game.(*Board).Step	0.01s	0.82%	1.18s	96.72%
runtime.semawakeup	0.01s	0.82%	0.01s	0.82%
internal/syscall/windows/registry.Key.Close	0	0.00%	0.01s	0.82%
internal/syscall/windows/registry.Key.GetMUISt	0	0.00%	0.02s	1.64%

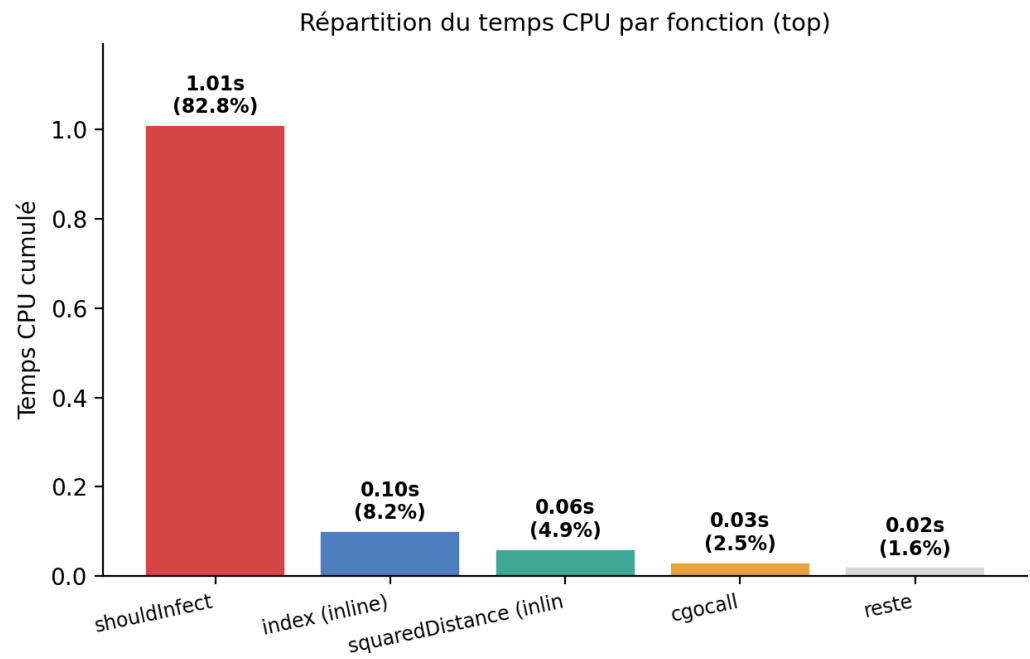


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

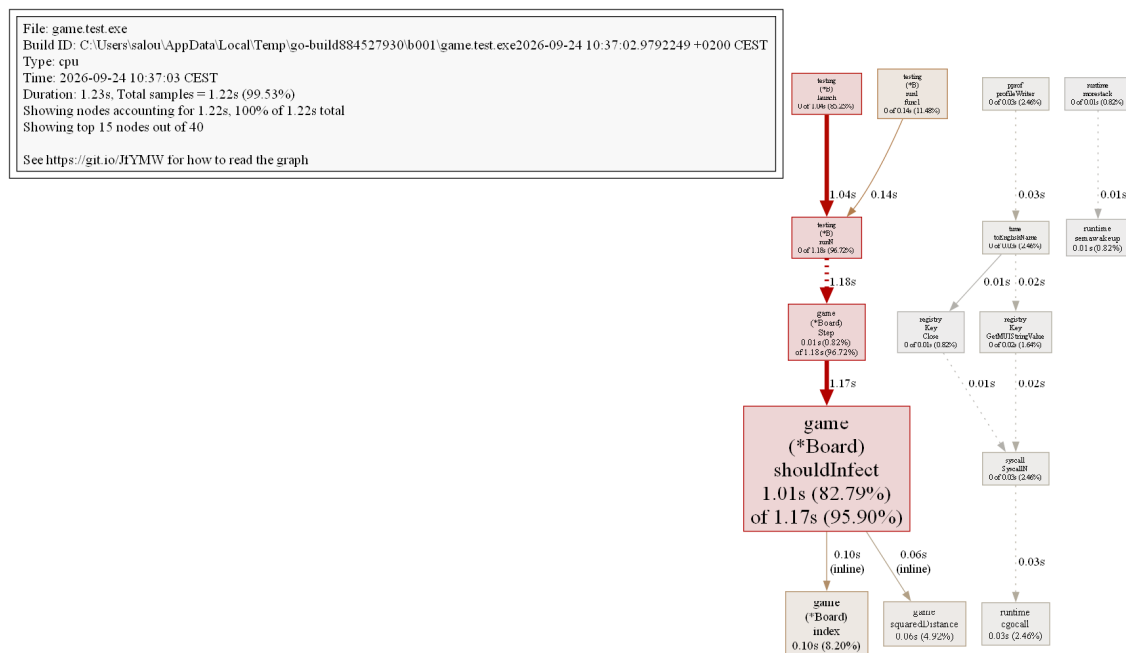


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **82.8%** du temps CPU total (flat), pour **95.9%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

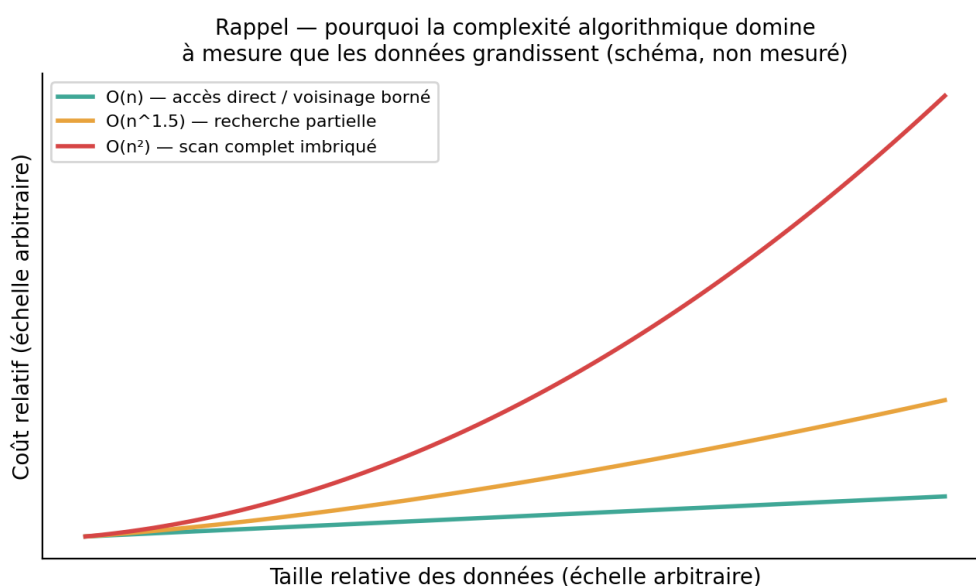


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **19812.81kB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard (inline)	6363.12kB	32.12%	6363.12kB	32.12%
jeu-de-la-vie/game.RandomBoard	5655.11kB	28.54%	12018.23kB	60.66%
runtime/pprof.StartCPUProfile	1184.27kB	5.98%	1184.27kB	5.98%
runtime.mallocgc	1026kB	5.18%	1026kB	5.18%
compress/flate.NewWriter (inline)	902.59kB	4.56%	1998.16kB	10.09%
compress/flate.newDeflateFast (inline)	583.01kB	2.94%	583.01kB	2.94%
runtime/pprof.(*protobuf).varint	513kB	2.59%	513kB	2.59%
flag.(*FlagSet).Var	512.88kB	2.59%	512.88kB	2.59%

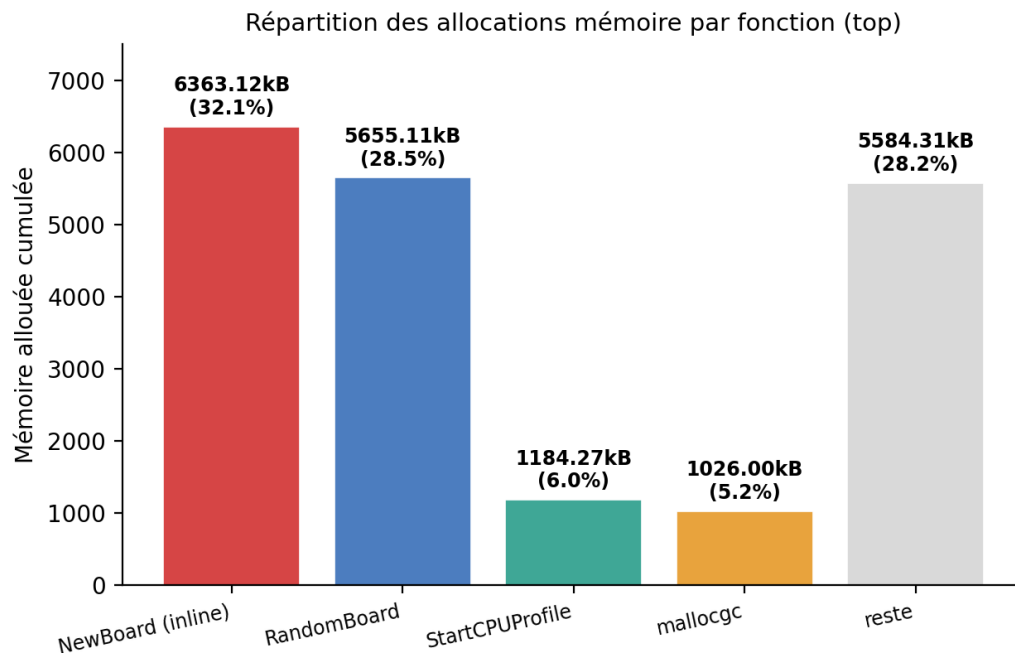
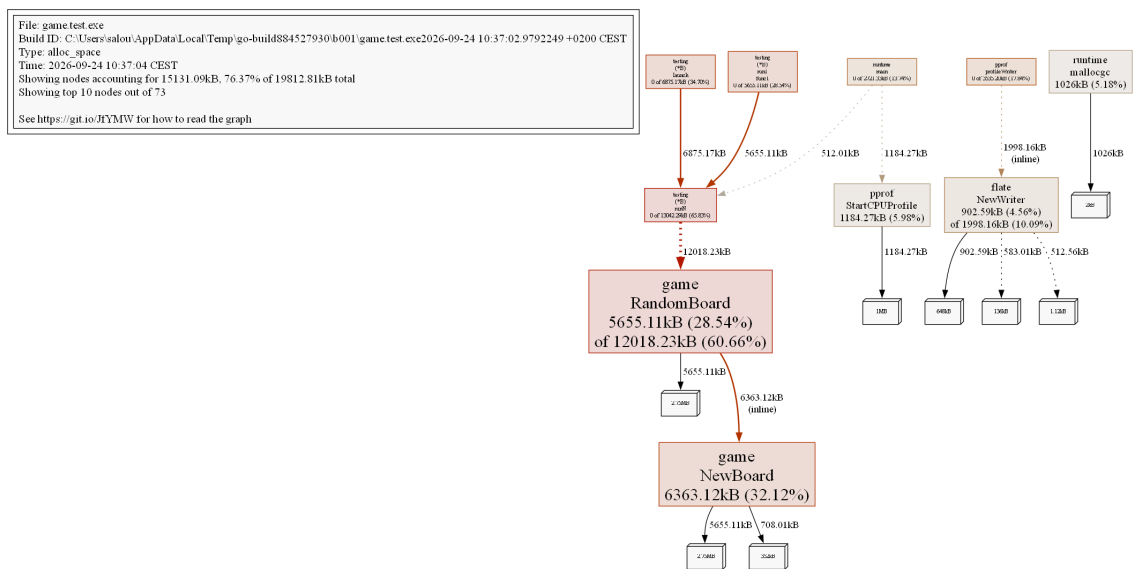


Figure — Répartition des allocations mémoire par fonction.



4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:37:13 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.